HW 1.

1. Pseudocode:

```
fib-it(n):
1   if n ≤ 2: return n  2
1   x = 0
1   y = 1
1   z
(n-1) for i = 2,...,n:
      2   z = x+y
      1   x = y
      1   y = z
1   return z
```

Claim: The algo fib-it is correct

Proof: We show by loop invariant that for any $n \in \mathbb{N}$, fib-it(n) returns F_n. For $n=0,1$ the correct val is returned w/out any iterations of the for loop. For $n \geq 2$ we'll argue that if any iteration $i \in \{2,\dots,n\}$

1. a precondition is that $x = F_{i-2}$ and $y = F_{i-1}$
2. a postcondition is that $z = F_i$

If they hold for i in general, then the postcondition for i and precond for i+1 follows from $z = x+y$ and that after this $x = y$ and $y = z$ so for the next loop, the $z = x+y$ equation is set up again. Upon termination, post condition for $i = n$ guarantees return val is F_n.

Observations

a. asymptotic runtime is $\underbrace{c_{mp} + c_{alloc} + 2c_{write} + (n-1)(2c_{read} + c_{add} + c_{write}) + c_{return}}_{\Theta(n) \text{ b/c } T_{add}(n) = \Theta(\log(n))}$

$$T_{fib\text{-}it}(n) = \begin{cases} 2 & \text{if } n < 2 \\ \Theta(n^2) & \text{if } n \geq 2 \end{cases} \quad T_{fib\text{-}it}(n) = \begin{cases} c_{compare} + c_{return} \\ c_{comp} + c_{alloc} + 2c_{write} + (n-1)(2c_{read} + c_{add} + c_{write}) + c_{return} \end{cases}$$

It's much faster than recursion, and even a little faster than the array ($4n+1$ vs $5n$). All measured runtimes stack up pretty well based on what we expected

b. There was no measured effect b/c both runtimes were so low. The optimized runtime vs og is $4n+1$ vs $5n$

c. The space complexity of the OG is $O(n)$ while the optimized version is $O(3)$ or constant.

$$f_{\text{fib-optimized}}(n) = \begin{cases} 0 & \text{if } n<2 = \Theta(1) \\ 4(3)=12 & \text{if } n \geq 2 = \Theta(1) \end{cases}$$ one int = 4 bytes

$$f_{\text{fib-unoptimized}}(n) = \begin{cases} 0 & \text{if } n<2 = \Theta(1) \\ 4(\lceil n \rceil \log(n)) & \text{if } n \geq 2 = \Theta(n \log n) \end{cases}$$

d. At $n=47$, the outputs went negative b/c $F_{47} > 2^{31}$ so our limit on positive ints was overrun (b/c we used the standart int data type). 2147483647 is the biggest positive number for ints which is less than F_{47}. To fix this, we could use longs or unsigned int, but the best is to combine both and use unsigned long long.

2. Claim: $F_n \geq 2^{n/2}$ for $n \geq 6$

Base cases: for $n=6$ and $n=7$ note that $F_6 = 8 \geq 8 = 2^{\frac{6}{2}}$ and $F_7 = 13 = \sqrt{169} \geq 2^{\frac{7}{2}} = \sqrt{128}$, establishing our base cases that $F_n \geq 2^{n/2}$ for $n=6,7$.

Inductive step: Fix $n \geq 8$ and assume $F_i \geq 2^{i/2}$ for all $i = 6, \ldots, n-1$. Note $F_n = F_{n-2} + F_{n-1}$ by definition, and $F_{n-2} \geq 2^{\frac{(n-2)}{2}}$ and $F_{n-1} \geq 2^{\frac{(n-1)}{2}}$ by assumption. Then $F_n \geq 2^{\frac{(n-2)}{2}} + 2^{\frac{(n-1)}{2}}$. $2^{\frac{(n-2)}{2}} + 2^{\frac{(n-1)}{2}} = 2^{\frac{(n-2)}{2}}(1+2^{\frac{1}{2}})$ and $2^{\frac{n}{2}} = 2^{\frac{(n-2)}{2}}(2)$ so w/ $2^{\frac{n-2}{2}} = 2^{\frac{n-2}{2}}$ and $(1+2^{\frac{1}{2}}) > 2$, $2^{\frac{(n-2)}{2}} + 2^{\frac{(n-1)}{2}} > 2^{\frac{n}{2}}$. B/c $F_n \geq 2^{\frac{n-2}{2}} + 2^{\frac{n-1}{2}}$, it is then also $\geq 2^{\frac{n}{2}}$

3. a. $f(n)$ $\quad$ $g(n)$

$n-100$ $\quad$ $n-200$

$\boxed{f = \Theta(g)}$

b. $f(n)$ $\quad$ $g(n)$ $\quad$ ....

$n^{\frac{1}{2}}$ $\quad$ $n^{\frac{2}{3}}$ $\quad$ $n^{\frac{2}{3}} > n^{\frac{1}{2}}$

$n^{\frac{1}{2}}$ vs $n^{\frac{1}{2}} \cdot n^{\frac{1}{6}} \rightarrow 1$ vs $n^{\frac{1}{6}}$ $\quad$ $\boxed{f = O(g)}$

$f(n)$ $g(n)$

c. $100n + \log n$ $n + (\log n)^2$

dominant term

can make O or Ω depending on const

$\boxed{f = \Theta(g)}$

d. $f(n) = n \log n$ $g(n) = 10n \log 10n$

$\boxed{f = \Theta(g)}$

e. $f(n) = \log(2n)$ $g(n) = \log(3n)$

$\boxed{f = \Theta(g)}$

f. $10 \log(n) = f(n)$ $g(n) = \log(n^2)$

$\boxed{f = \Theta(g)}$

g. $n^{1.01} = f(n)$ $g(n) = n \log^2(n)$

$\boxed{f = \Omega(g)}$

h. $f(n) = \frac{n^2}{\log n}$ $g(n) = n \log^2(n)$

$\boxed{f = \Omega(g)}$

just a weaker version of this

i. $f(n) = n^1$ $g(n) = \log(n)^{10}$

$\boxed{f = \Omega(g)}$

j. $\log(n)^{\log(n)} = f(n)$ $g(n) = \frac{n}{\log(n)}$

$f = \Omega(g)$ b/c it is still an exponential function vs g which is a non-const polynomial. Doesn't matter if the exp is $\log(n)$

k. $f(n) = \sqrt{n}$, $g(n) = (\log n)^3$

$\boxed{f = \Omega(g)}$

l. $f = n^{\frac{1}{2}}$ $g = 5^{\log_2 n}$

$\boxed{f = O(g)}$

m. $f = n2^n$ $3^n = g$

$\boxed{f = O(g)}$

n. $f = 2^n$, $g = 2^{n+1}$

$\boxed{f = \Theta(g)}$

o. $f = n!$ $g = 2^n$

$\boxed{f = \Omega(g)}$

$n! \leq (n+1)!$ and $2^n < 2^{n+1}$

when $n \geq 1$. for $n \geq 4$

$\therefore n! = 24$, $2^n = 16$

$24 > 16$ so $n! > 2^n$.

$(n+1)! = n!(n+1)$

$2^{n+1} = 2^n(2)$

we've proven $n! > 2^n$ for $n > 4$

and b/c $n > 4$ $(n+1) = 5$ which is > 2

b/c $5 > 2$ and $n! > 2^n$ for $n > 4$

$5n! > 2^n \cdot 2 \rightarrow (n+1)! > 2^{n+1}$

$\boxed{f = \Omega(g)}$

p. $\log n^{\log n} = f$ $g = 2^{\log_2 n^2}$

base two is irrelevant

$\log n^{\log n}$ vs $2^{(\log n)(\log n)}$

$\log(n)\log(n) > \log(n)$

$\log(n) > 2$

the expo matters most

$\boxed{f = O(g)}$

9.

$f = \sum_{i=1}^{n} i^k \qquad g = n^{k+1}$

$n^k + \ldots + 1^k$ vs n^{k+1}

just experimenting

$k = 3$

$n = 4 \qquad g = 256$

$f = 64 + 27 +$

$8 + 1 \qquad \frac{256}{100} = 2.56$

$k = 4$

$n = 5 \qquad g = 3125$

$f = 625 + 256 + 81 +$

$16 + 1 \; 2979$

$\frac{3125}{979} = 3.19$

ratio is getting larger

n^{k+1} is growing faster

$\boxed{f = O(g)}$